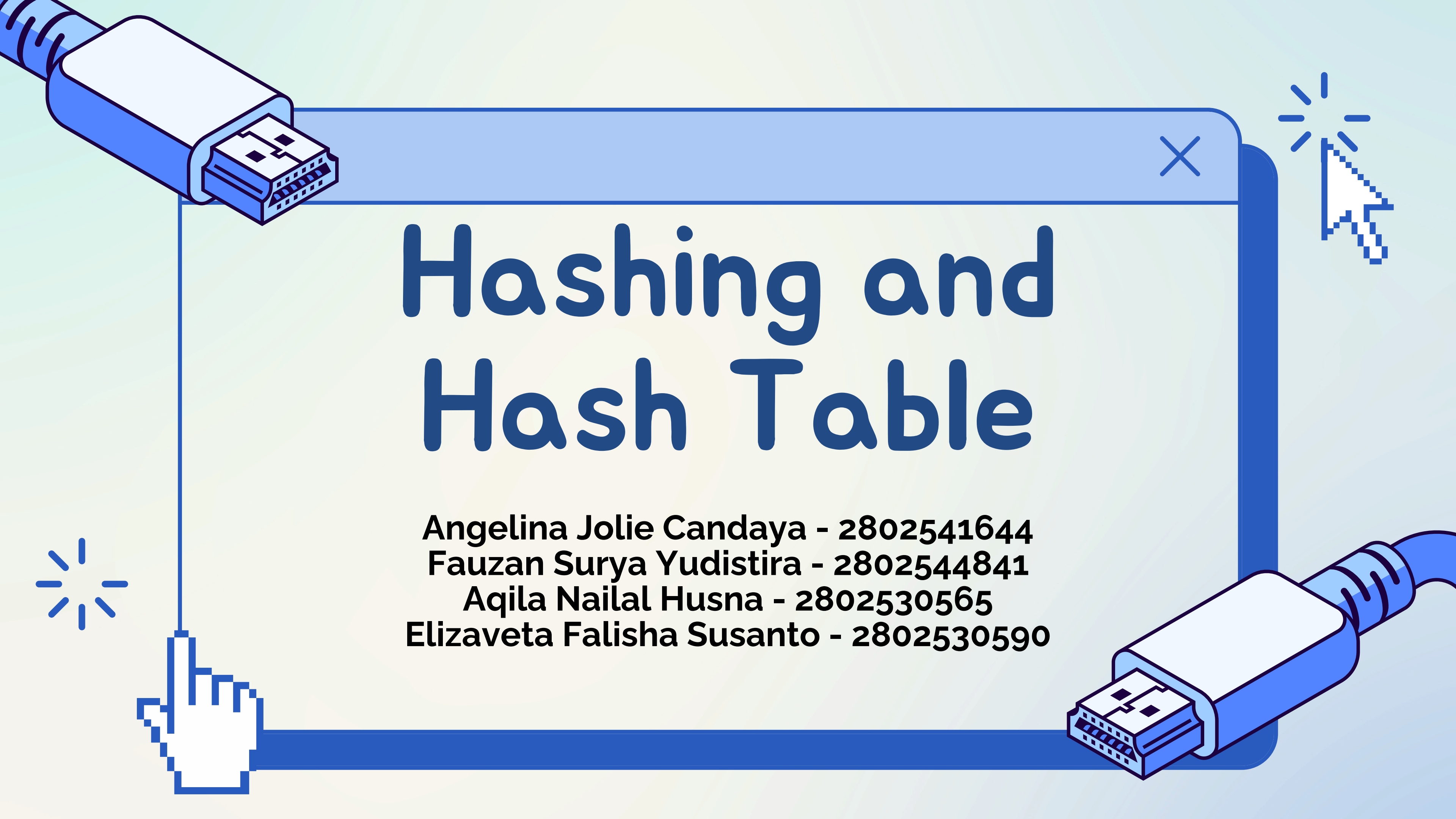
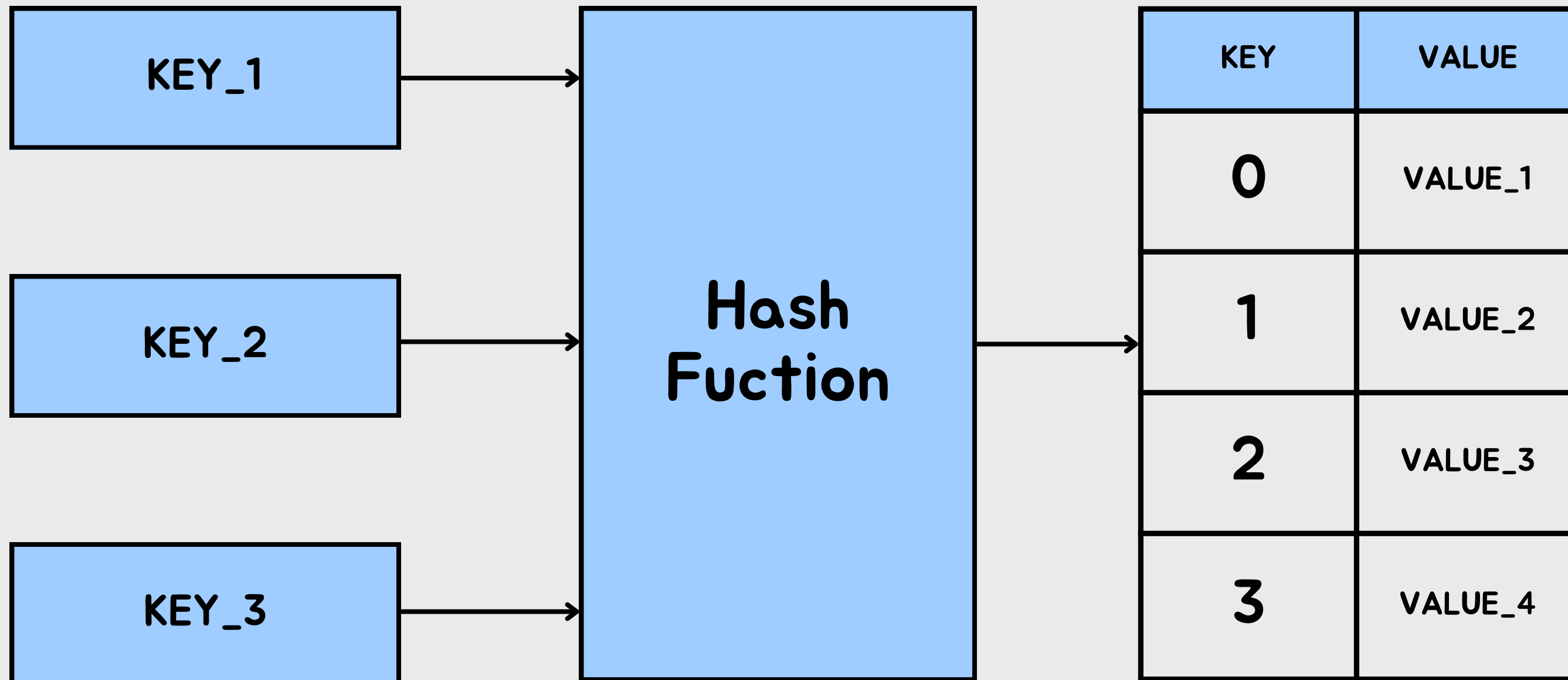




Hashing and Hash Table

Angelina Jolie Candaya - 2802541644
Fauzan Surya Yudistira - 2802544841
Aqila Nailal Husna - 2802530565
Elizaveta Falisha Susanto - 2802530590



Key

unique identity used to find specific data

Value

information stored in relation to a key

Hash

a mathematical function that converts a key to an index in an array

Key

Value

"Orange"

→ Fruit, Color

"z"

→ Last alphabet

"081234567899"

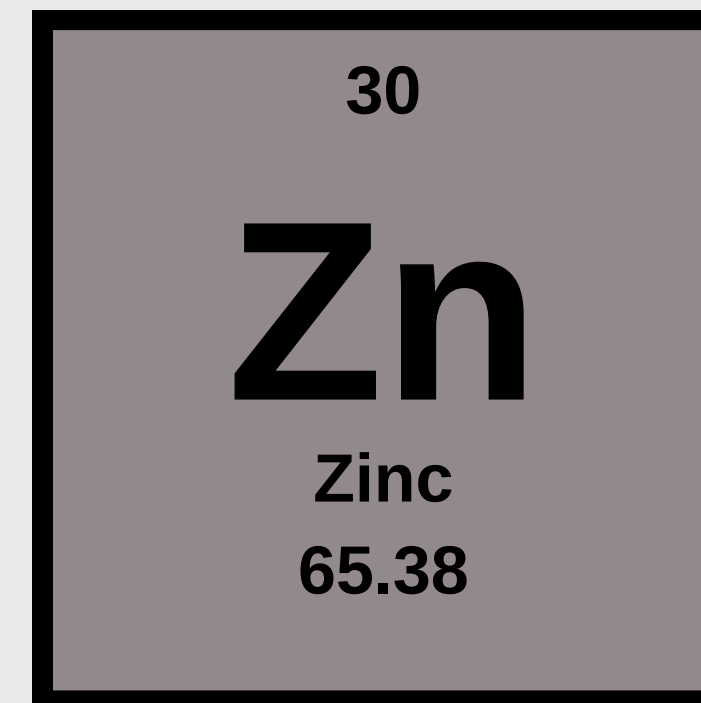
→ Phone number

"4"

→ Number 4, "death"

"25"

→ Number 25





Key: 1, 2, 3, 5, 7, 10

$h(k) = k$

0	1	2	3	4	5	6	7	8	9
	1	2	3		5		7		

Key: 11, 23, 30, 57, 78

$h(k) = k \bmod \text{size}$

0	1	2	3	4	5	6	7	8	9
30	11		23				57	78	

Key: 1, 5, 10, 15, 22

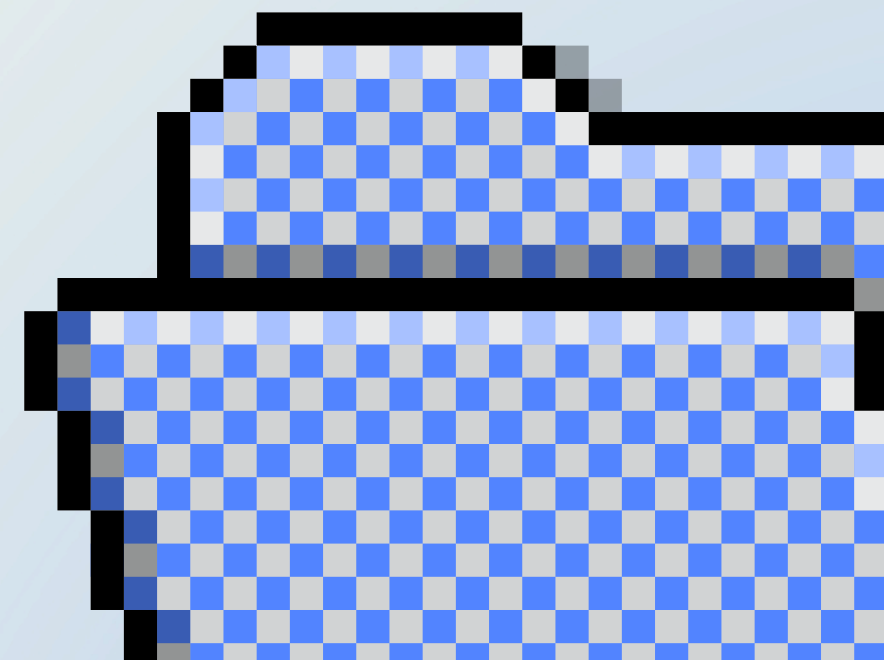
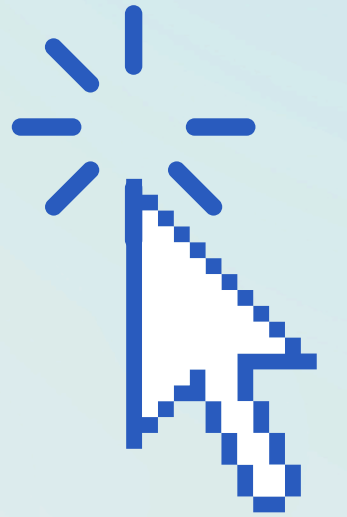
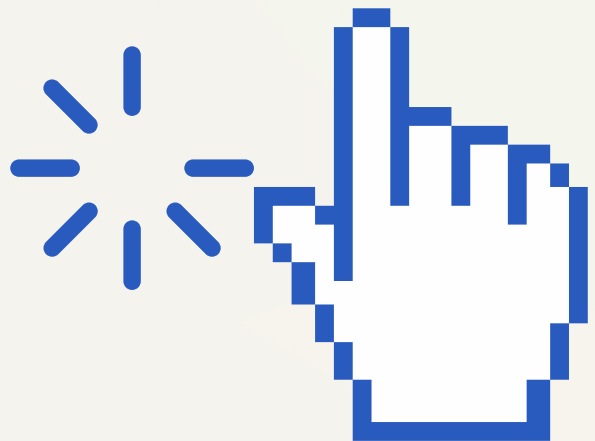
$h(k) = k \bmod \text{size}$

0	1	2	3	4	5	6	7	8	9
10	1	22			5				

Hash Function

How to map keys to indexes:

1. **Mid Square**
2. **Folding**
3. **Division**
4. **Digit Extraction**
5. **Rotating Hash**

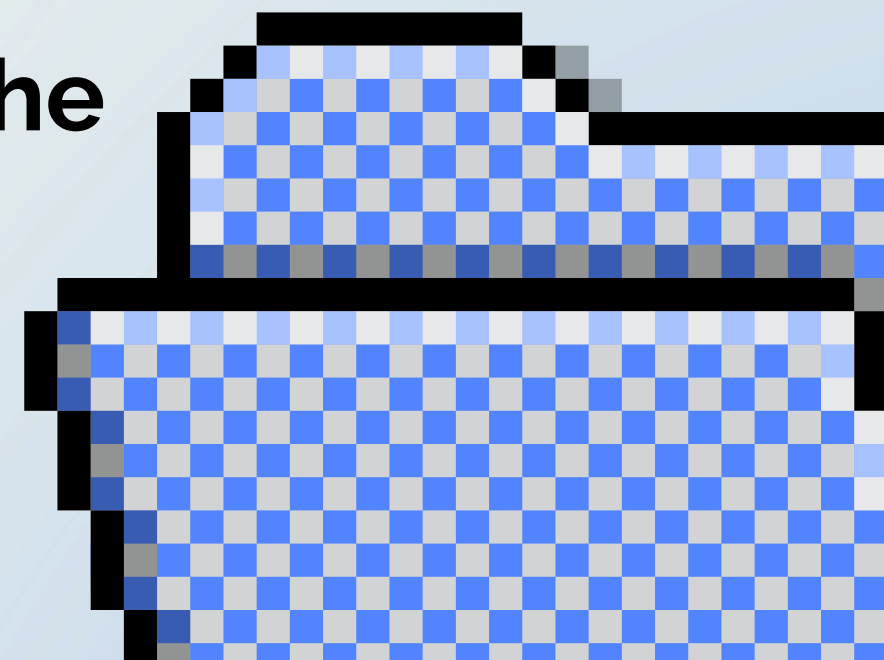
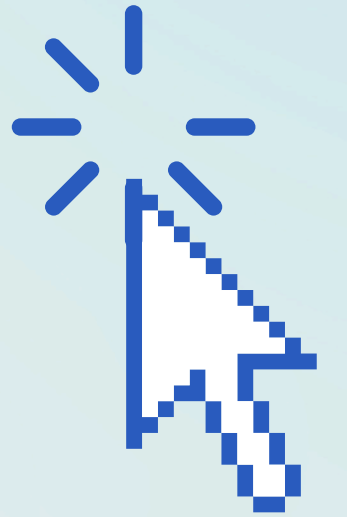
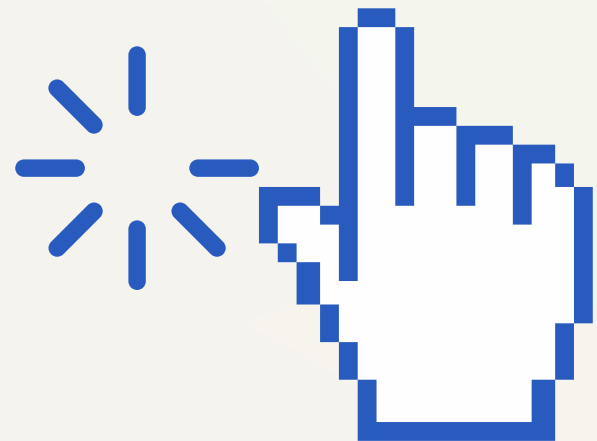


Mid-Square

This method works by squaring the key value and taking the middle part of the squared result as the hash.

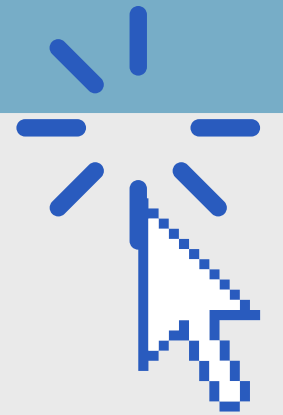
Steps:

1. Square key (k^2)
2. Take bits from the middle part of the squared result





MID-SQUARE



Function: $h(k) = s$

- **k = key (the value to be hashed)**
- **s = hash key (the result of squaring the key)**

Even if two keys are similar, their squared results still differ significantly.

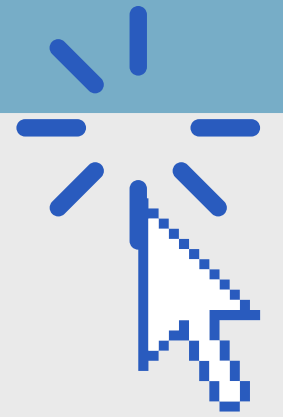
For example, let's say we have these keys:

- **3121, squared result 9740641, middle part 406.**
- **3122, squared result 9746884, middle part 468.**
- **3123, squared result 9753129, middle part 531.**

Although the key values differ only slightly, the resulting hashes are still different.



MID-SQUARE



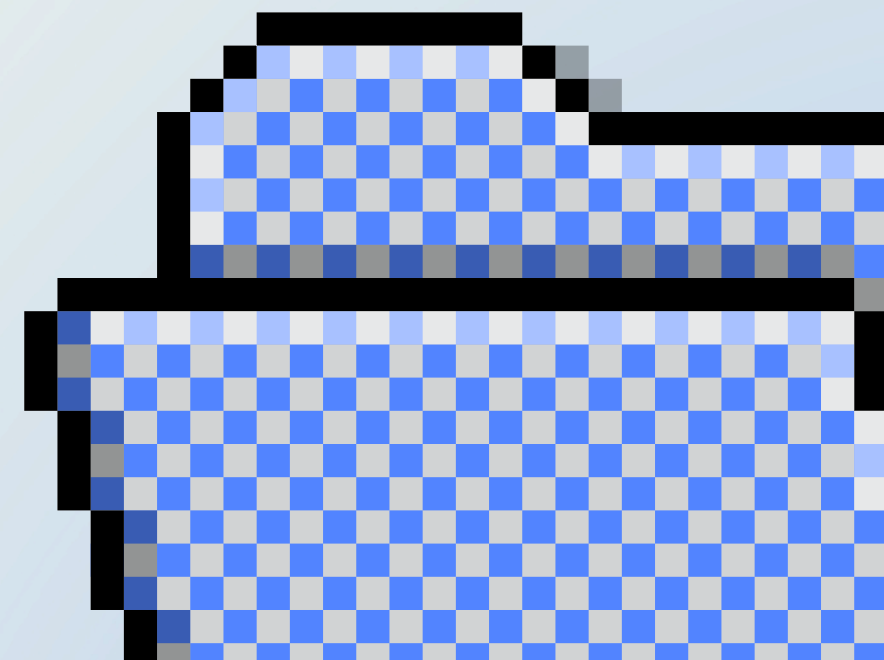
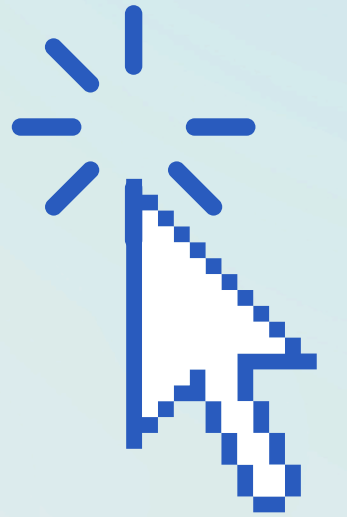
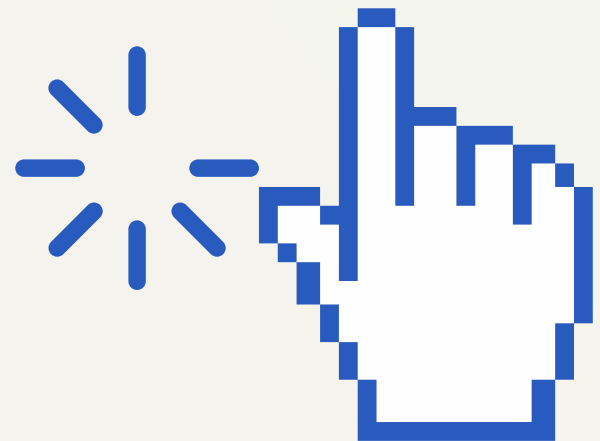
- The hash table size is preferably a power of 2 (as it is more efficient).
- If the chosen table size is 1024, the binary of the square of 3121 is **1001010-0101000010-1100001**

Folding

This method divides the key into several smaller parts, then adds up those parts to obtain the hash.

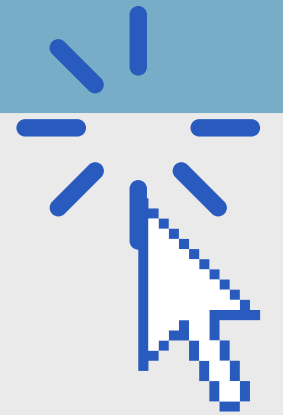
Steps:

- 1. Divide the key into smaller parts**
- 2. Add each part together**





FOLDING



Example: Given a hash table with 100 locations, calculate the hash values of 5678, 321, and 34567

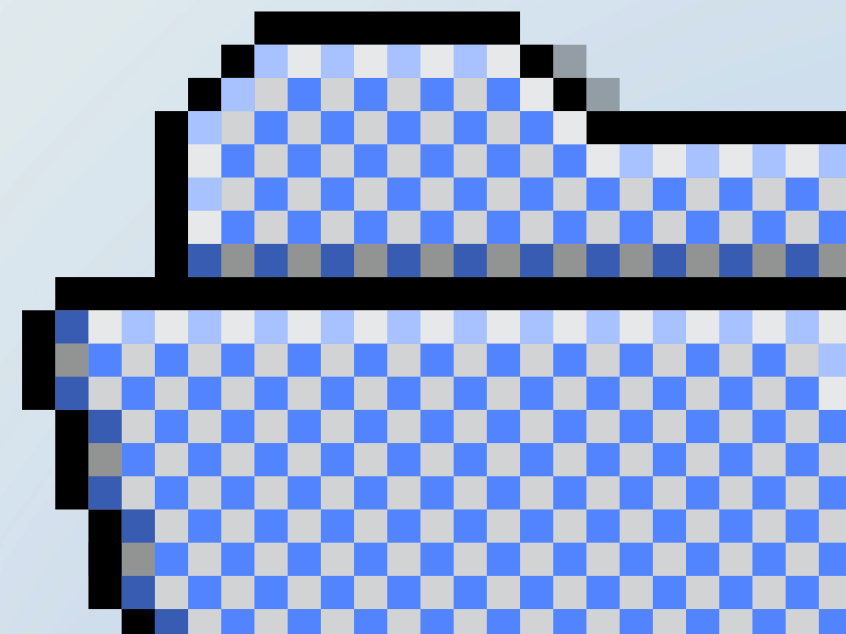
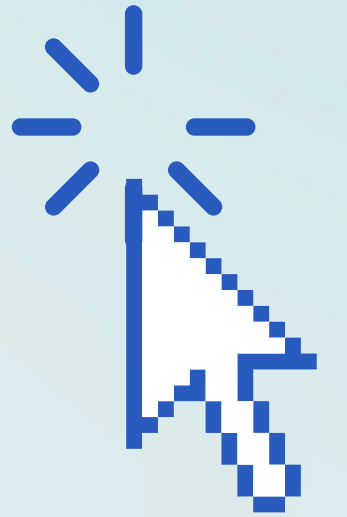
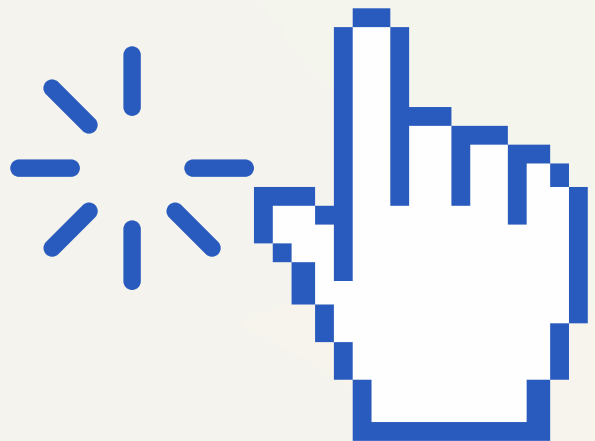
Key	Split Parts	Sum	Mod 100	Hash Value
5678	56 & 78	134	$134\%100 = 34$	34
321	32 & 1	33	$33\%100 = 33$	33
34567	34, 56, & 7	97	$97\%100 = 97$	97

Division

This method uses the modulo of the key with the size of the hash table.

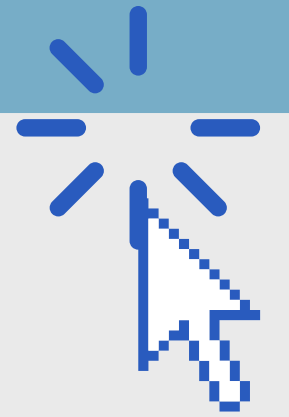
Steps:

1. Given a key then find the modulo by a value (prime number, hash table size, or memory size used).





DIVISION METHOD



Function $h(z) = z \bmod M$

$z = \text{key}$

M = value to divide key (prime number, table size, or memory)

Key: 55, 12, 8, 41, 19

Table size: 5

Modulo every key with table size = 5

$55 \bmod 5 = 0$ $41 \bmod 5 = 1$

$12 \bmod 5 = 2$ $19 \bmod 5 = 4$

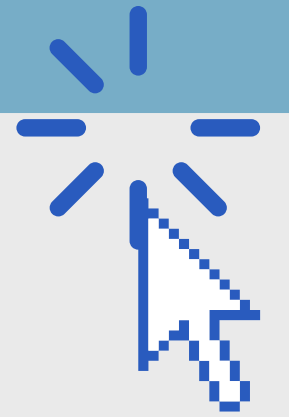
$8 \bmod 5 = 3$

$h \{ \}$	Key
0	55
1	41
2	12
3	8
4	19





DIVISION METHOD



for strings we need to sum the ASCII of each character and then modulo the size of the hash table.

example: insert {"CAT", "DOG", "ANT"} stored in hash size 5

CAT -> $67 + 65 + 84 = 216 \rightarrow 216 \% 5 = 1$

DOG -> $68 + 79 + 71 = 218 \rightarrow 218 \% 5 = 3$

ANT -> $65 + 78 + 84 = 227 \rightarrow 227 \% 5 = 2$

h {}	Key
0	
1	CAT
2	ANT
3	DOG
4	

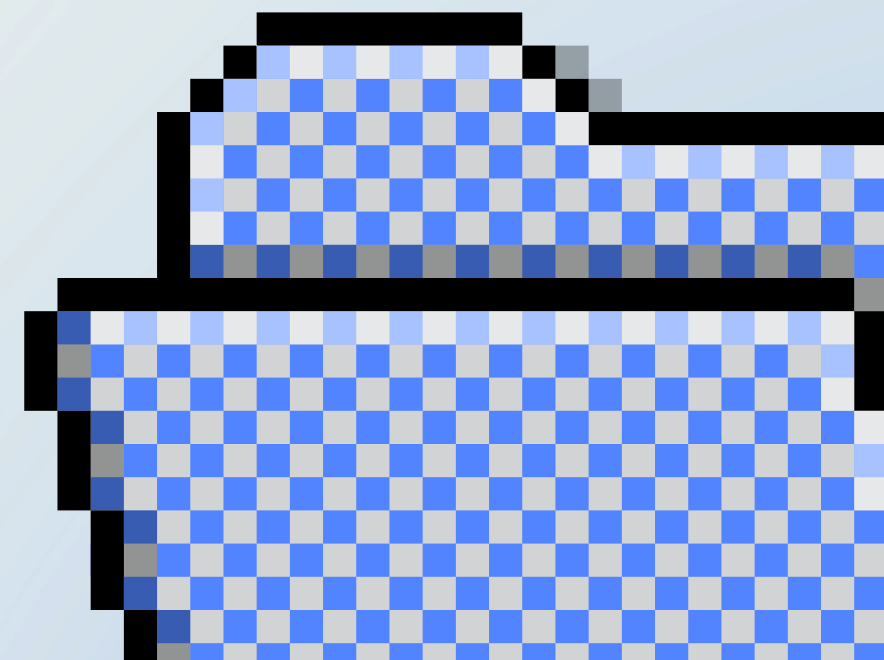
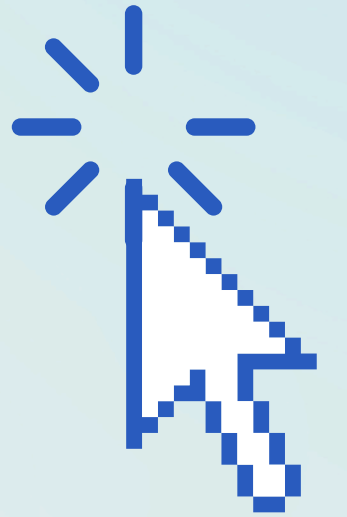
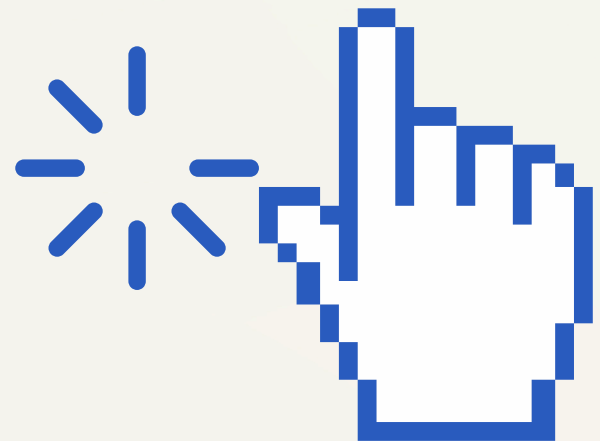


Digit Extraction

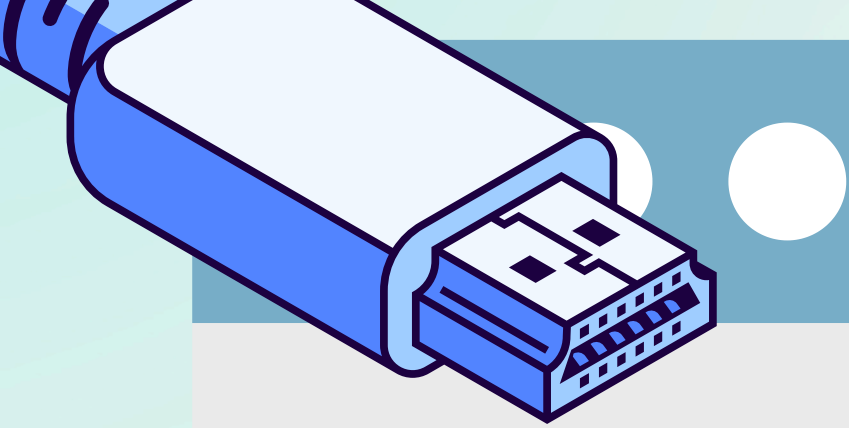
This method determines a hash address based on extracting specific digits from a given number (key).

Steps:

1. Given a number (key)
2. Determine which digits to extract
3. Combine extracted digits



DIGIT EXTRACTION



Given

$x = 14,568$

- **Select specific digits based on the position in the number**

first position -> 1

third position -> 5

fifth position -> 8

- **Combine those numbers**

Hash code = 158

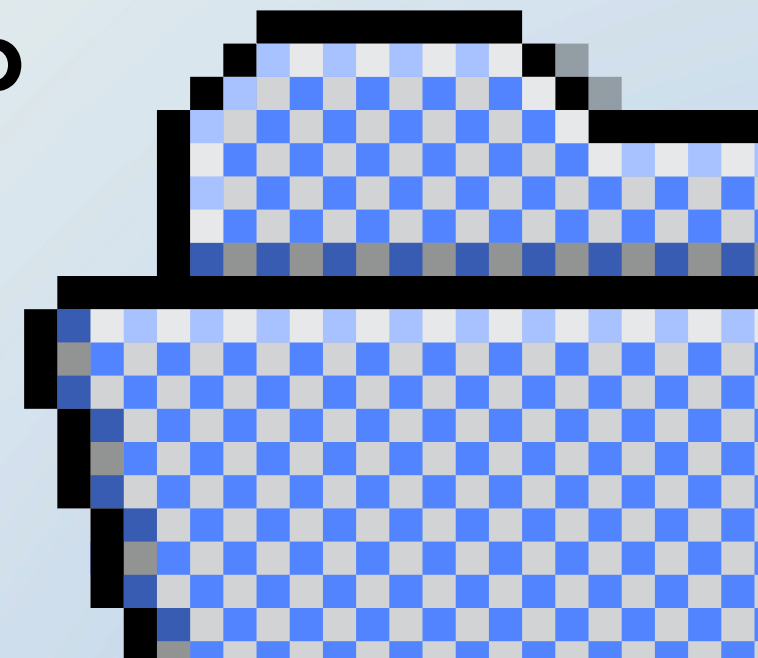
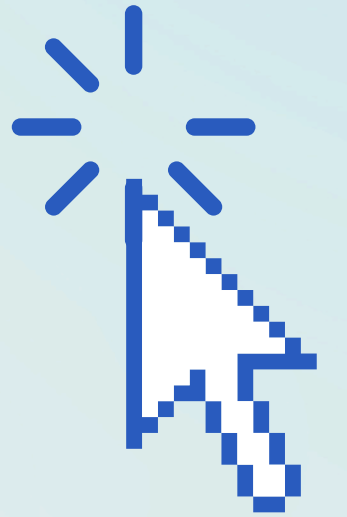
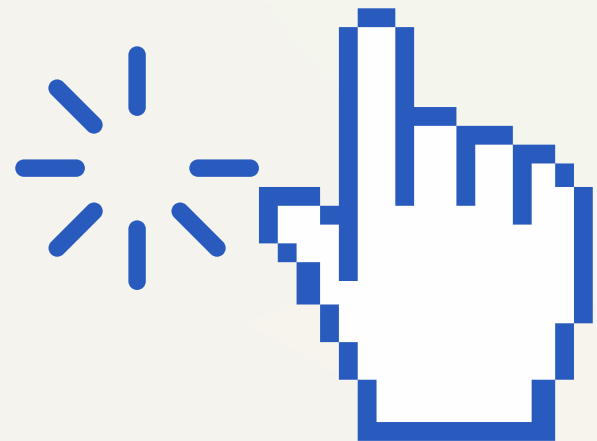


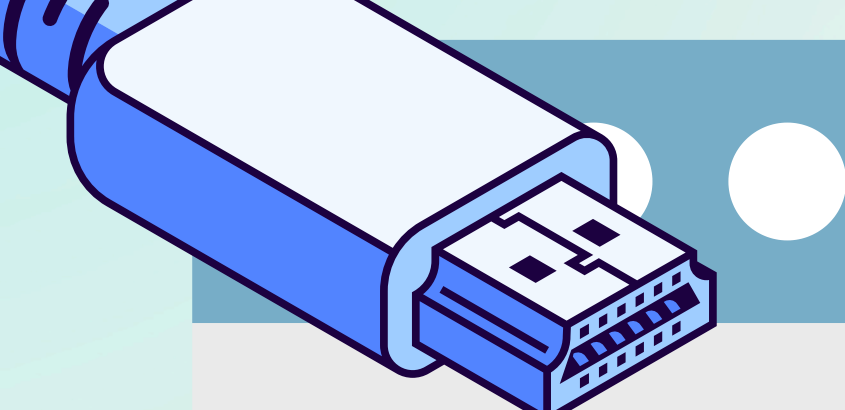
Rotating Hash

This method applies shifting the digits of the hash address to obtain a new hash address.

Steps:

- 1. Get the initial hash address using the division or mid-square method**
- 2. Perform rotating by shifting the hash address to get a new hash address**





ROTATING HASH



Given

hash address = 20021

- **Do rotating**

rotation result = 12002

- **For placement in the hash table index, you can use division (modulo with size) or mid-square.**



Collision

Collision: A collision occurs when two or more keys in a hash table produce the same index or hash value.

Example:

Key = 15, 50
Table size = 5

$$15 \bmod 5 = 0$$

$$50 \bmod 5 = 0$$

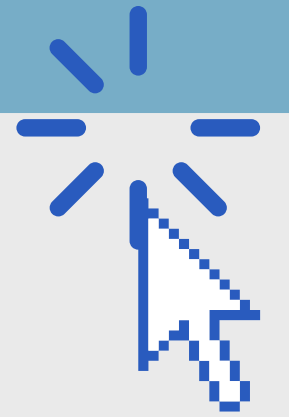
Two Methods to Handle Collisions:

1. **Open Addressing:** linear probing & rehashing
2. **Chaining**





1. Open Addressing: Linear Probing



How it works?

When a collision occurs (i.e., two keys hash to the same index), linear probing searches for the next available slot by checking the subsequent indices one by one in a linear fashion.

Example:

Key: 10, 21, 18, 15, 20

Table size: 5

Modulo each key with table size = 5

$10 \bmod 5 = 0$ $15 \bmod 5 = 0$

$21 \bmod 5 = 1$ $20 \bmod 5 = 0$

$18 \bmod 5 = 3$

h {}	Key
0	10
1	21
2	15
3	18
4	20





1. Open Addressing: Rehashing



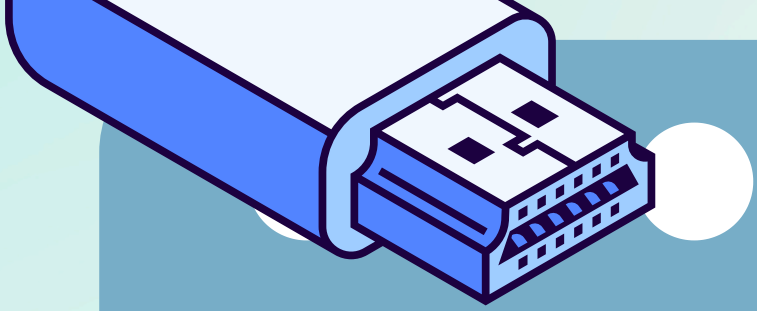
How it works?

Creating a new hash table with double the size of the original table. All entries in the original hash table must then be rehashed and transferred to the new table.

0	1	2	3	4
	26	31	43	17

0	1	2	3	4	5	6	7	8	9
	31		43			26	17		





2. Chaining



How it works?

When a collision occurs, the new key is simply added to the list at that index (linked list concept), rather than searching for another empty slot.

Example:

Key: 10, 21, 18, 15, 20

Table size: 5

Modulo each key with table size = 5

$10 \bmod 5 = 0$ $15 \bmod 5 = 0$

$21 \bmod 5 = 1$ $20 \bmod 5 = 0$

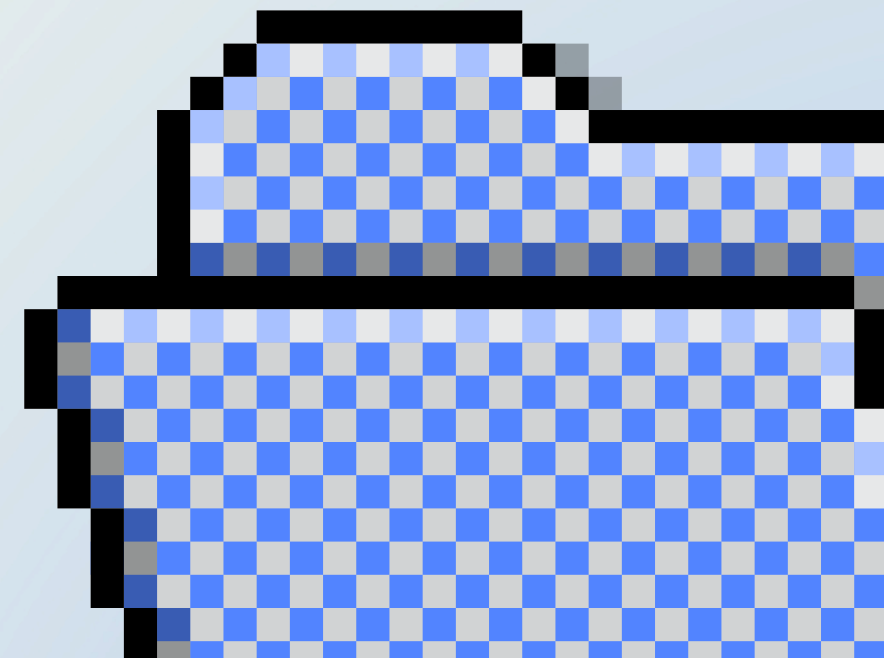
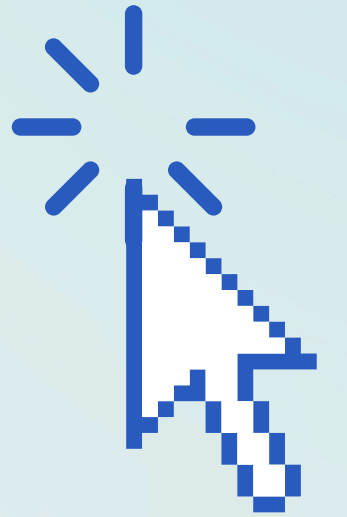
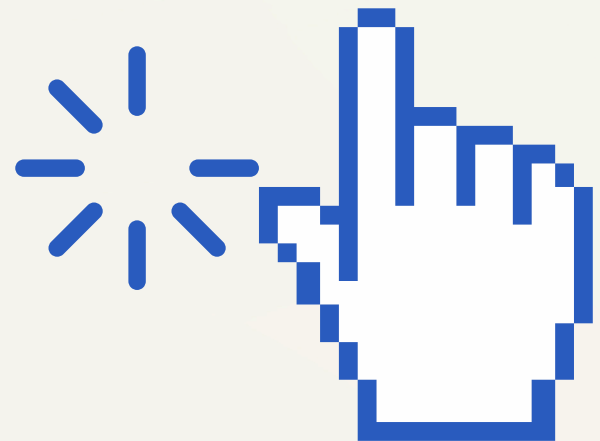
$18 \bmod 5 = 3$

h {}	Key
0	10 -> 15 -> 20
1	21
2	
3	18
4	



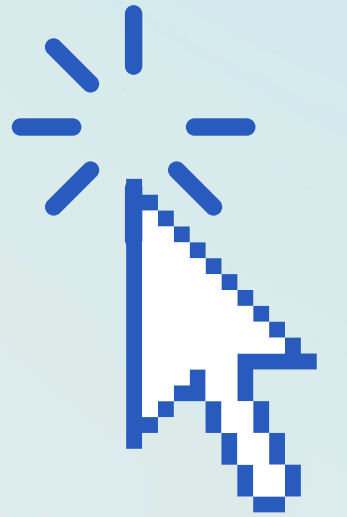
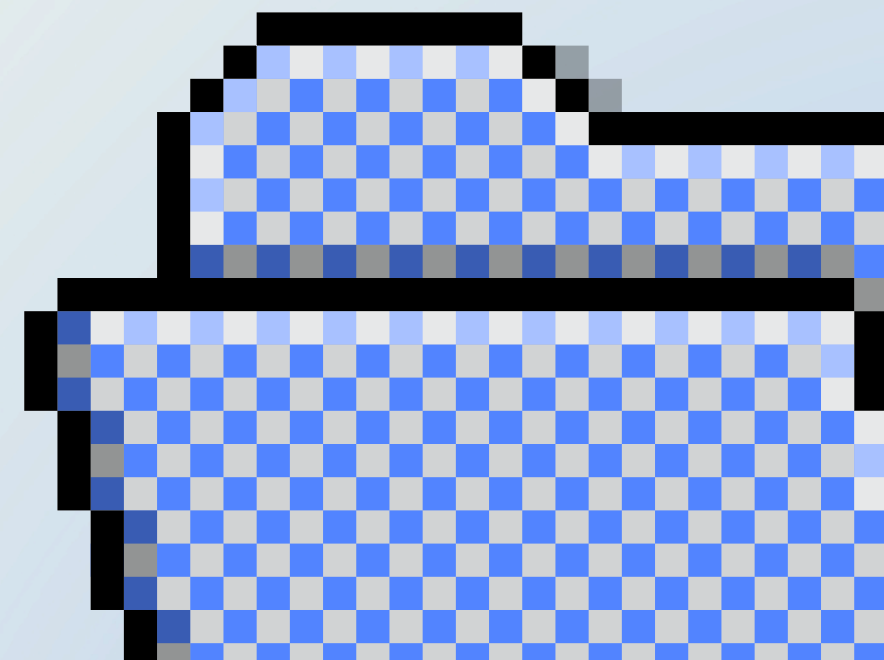
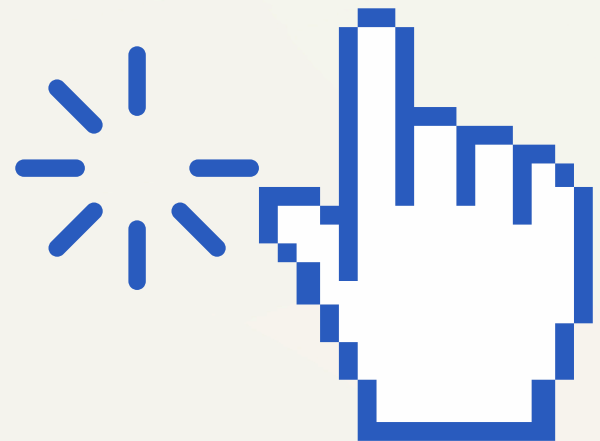
Advantage

1. **Extremely Fast Data Lookup, Insertion, and Deletion (Average $O(1)$)**
2. **Memory Efficiency**
3. **Flexible for Various Key Types**
4. **Relatively Simple Implementation**



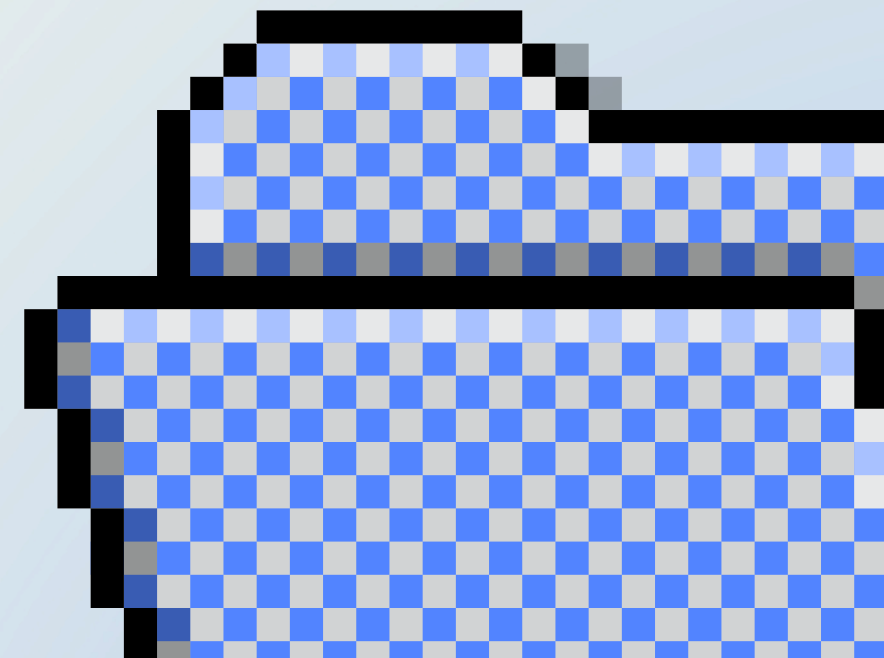
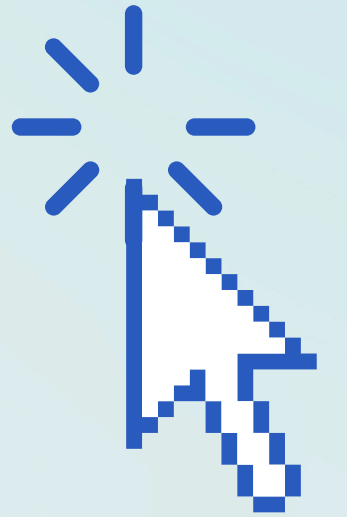
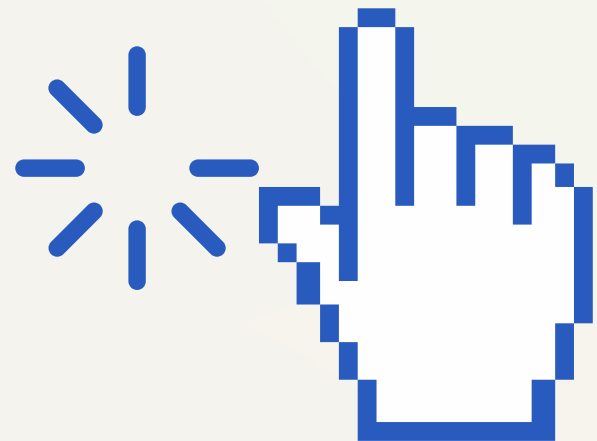
Disadvantage

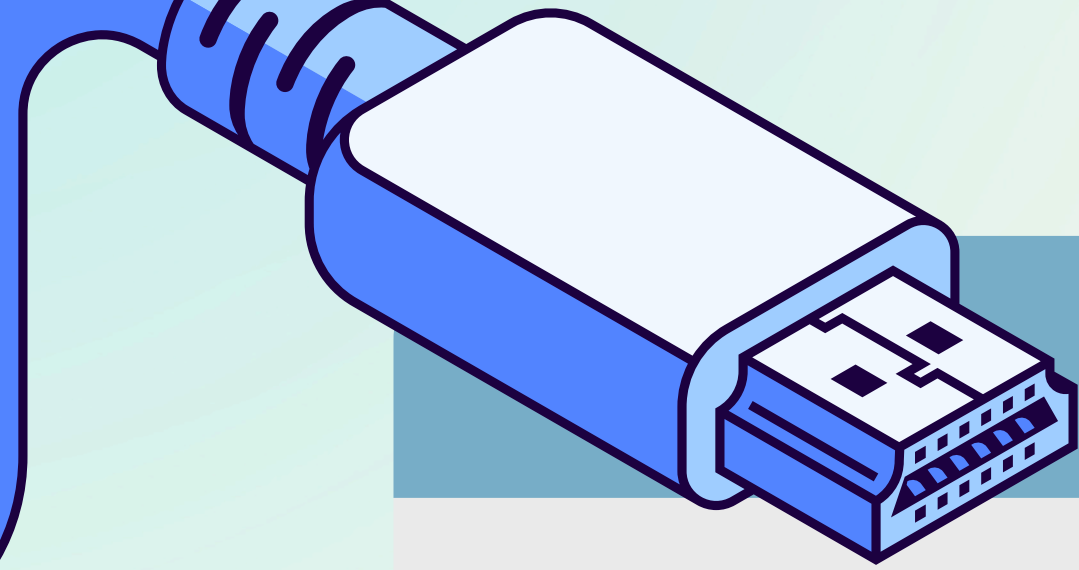
1. **Potential for Collisions**
2. **Does Not Support Ordered Traversal of Elements**
3. **Initial Hash Table Size Needs to Be Well-Planned**



Real-Life Applications

1. **Password Storage (Hashing passwords)**
2. **Dictionary / Spell Check**
3. **Caching (Web browser, memory cache)**
4. **Databases and Indexing**
5. **Symbol Tables in Compilers**
6. **Search Engines (Keyword indexing)**





THANK YOU

See You Tomorrow!

